

# Mathematical Foundations of Kafka Producers & Consumers

---

## Companion to Session 2 | Big Data Engineering Exercise Sheet (without solutions)

Reliability, Delivery Semantics, and Throughput Analysis  
for Producers, Consumers, and the Durability Triangle

### Document Contents

- Probability of message loss for each `acks` level.
- Geometric model of retries and the idempotent producer.
- Delivery semantics formalised: at-most-once, at-least-once, exactly-once.
- Batching trade-off: throughput vs. latency under `linger.ms`.
- Consumer-group dynamics and lag analysis.
- **Two exercises (statements only)** to be solved by the student.

## Contents

---

|          |                                                                     |          |
|----------|---------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>2</b> |
| <b>2</b> | <b>Producer Reliability: acks as a Bernoulli Model</b>              | <b>2</b> |
| 2.1      | The acknowledgement chain . . . . .                                 | 2        |
| 2.2      | Numerical comparison . . . . .                                      | 3        |
| <b>3</b> | <b>Retries: The Geometric Model</b>                                 | <b>3</b> |
| 3.1      | Why retries help . . . . .                                          | 3        |
| 3.2      | Probability of giving up . . . . .                                  | 4        |
| 3.3      | Expected latency under retries . . . . .                            | 4        |
| 3.4      | The role of idempotence . . . . .                                   | 4        |
| <b>4</b> | <b>Delivery Semantics: a <math>2 \times 2</math> Classification</b> | <b>4</b> |
| 4.1      | Two binary failure modes . . . . .                                  | 4        |
| 4.2      | Mapping to commit strategy . . . . .                                | 4        |
| 4.3      | Probability of duplicates under at-least-once . . . . .             | 5        |
| <b>5</b> | <b>Batching: Throughput vs. Latency</b>                             | <b>5</b> |
| 5.1      | Why batching exists . . . . .                                       | 5        |
| 5.2      | Effective throughput . . . . .                                      | 5        |
| 5.3      | Numerical illustration . . . . .                                    | 6        |
| <b>6</b> | <b>Consumer Groups: Partition Assignment Combinatorics</b>          | <b>6</b> |
| 6.1      | The assignment constraint . . . . .                                 | 6        |
| 6.2      | Lag dynamics . . . . .                                              | 6        |
| 6.3      | Catch-up time . . . . .                                             | 6        |
| <b>7</b> | <b>Exercises</b>                                                    | <b>7</b> |
| <b>8</b> | <b>Summary</b>                                                      | <b>9</b> |
| 8.1      | Going further . . . . .                                             | 9        |

## 1. Introduction

Session 2 is built around three families of design choices: *producer acknowledgements*, *consumer offset commits*, and *durability configuration*. Each choice corresponds to a distinct probabilistic model. The goal of this companion document is to give those models a precise mathematical formulation so that you can compare configurations *quantitatively* rather than only qualitatively.

We focus on five building blocks:

1. **Producer reliability** as a Bernoulli model  $\rightarrow$  probability of message loss for each `acks` level.
2. **Retries** as a geometric trial  $\rightarrow$  expected number of attempts and tail latency.
3. **Delivery semantics** as a  $2 \times 2$  table  $\rightarrow$  at-most-once vs. at-least-once vs. exactly-once.
4. **Batching** via Little’s Law  $\rightarrow$  how `linger.ms` amortises network cost.
5. **Consumer-group rebalancing**  $\rightarrow$  partition assignment combinatorics and lag dynamics.

## 2. Producer Reliability: `acks` as a Bernoulli Model

### 2.1 The acknowledgement chain

Recall that a Kafka producer can be configured with three durability levels: `acks=0`, `acks=1`, `acks=all`. Each value defines how many brokers must persist a record before the producer considers it “sent”. We can model the loss of a single record as the joint failure of the brokers in the acknowledgement chain.

#### Definition – Bernoulli Failure Model

Let each broker fail independently with probability  $p \in [0, 1]$  during the short time window of a single write. Let  $r$  be the partition replication factor and  $m$  the value of `min.insync.replicas`. Then the probability of losing a single record is:

$$\mathbb{P}(\text{loss} \mid \text{acks} = v) = \begin{cases} 1 - (1 - p) \cdot \mathbb{P}(\text{later persisted}) & v = 0, \\ p & v = 1, \\ p^m & v = \text{all}. \end{cases}$$

The intuition behind each row:

- **`acks=0`**: the producer fires-and-forgets. The record is lost if it never reaches the leader, or reaches the leader but the leader crashes before flushing. We treat this as a fixed loss probability  $\geq p$ .

- **acks=1**: the leader writes locally and acknowledges. Loss occurs if the leader crashes *after* acknowledging but *before* the followers replicate. The marginal failure probability is  $\approx p$ .
- **acks=all**: the producer waits until all in-sync replicas have written. Loss requires *every* ISR member to fail before the next replica catches up. Independence yields  $p^m$ .

## 2.2 Numerical comparison

For  $p = 0.01$  (each broker has 1% failure probability per write window):

| acks | $m$ | Loss probability | Expected losses per $10^6$ records |
|------|-----|------------------|------------------------------------|
| 0    | —   | $\sim 10^{-2}$   | $\sim 10\,000$                     |
| 1    | —   | $10^{-2}$        | 10 000                             |
| all  | 1   | $10^{-2}$        | 10 000                             |
| all  | 2   | $10^{-4}$        | 100                                |
| all  | 3   | $10^{-6}$        | 1                                  |

### Important Remark

The pair (**acks=all**, **min.insync.replicas=2**) is the standard production recommendation. It pushes loss probability from  $10^{-2}$  to  $10^{-4}$  at the cost of a few milliseconds of additional latency.

## 3. Retries: The Geometric Model

### 3.1 Why retries help

Network hiccups and transient broker errors cause individual write attempts to fail. The Kafka producer automatically retries up to **retries** times with a back-off of **retry.backoff.ms** between attempts.

Let each individual attempt fail independently with probability  $q$ . The number of attempts  $T$  until the first success is a **geometric random variable**:

$$\mathbb{P}(T = k) = (1 - q) \cdot q^{k-1}, \quad k = 1, 2, 3, \dots$$

### **X<sup>1</sup>** Theorem – Expected Number of Attempts

For a geometric distribution with success probability  $1 - q$ :

$$\mathbb{E}[T] = \frac{1}{1 - q}, \quad \text{Var}(T) = \frac{q}{(1 - q)^2}.$$

### 3.2 Probability of giving up

Suppose the producer is configured with  $R$  maximum retries (so up to  $R + 1$  total attempts). The probability that *all* attempts fail is

$$\mathbb{P}(\text{still failing after } R + 1 \text{ attempts}) = q^{R+1}.$$

This is the residual loss probability after the retry policy is applied.

### 3.3 Expected latency under retries

If each attempt takes  $L_0$  milliseconds and the back-off is  $B$  ms, the expected total time to deliver a successful record is

$$\mathbb{E}[\text{latency}] = \mathbb{E}[T] \cdot L_0 + (\mathbb{E}[T] - 1) \cdot B.$$

For  $q = 0.05$  (5% per-attempt failure rate),  $L_0 = 10$  ms,  $B = 100$  ms:

$$\mathbb{E}[T] = \frac{1}{0.95} \approx 1.053, \quad \mathbb{E}[\text{latency}] \approx 1.053 \cdot 10 + 0.053 \cdot 100 \approx 15.8 \text{ ms}.$$

The mean latency is barely affected, but the *tail* is dominated by the back-off. This is why aggressive retries can spike  $p_{99}$  latency.

### 3.4 The role of idempotence

Without idempotence, each retry can produce a *duplicate* on the broker side. The probability that a given record is duplicated at least once is

$$\mathbb{P}(\text{duplicate}) = 1 - \mathbb{P}(\text{first attempt succeeds}) = 1 - (1 - q) = q.$$

Idempotent producers eliminate this risk: the broker tracks a sequence number per producer/partition pair and discards duplicates. The duplication probability drops to 0.

## 4. Delivery Semantics: a $2 \times 2$ Classification

---

### 4.1 Two binary failure modes

Every messaging system is characterised by two independent failure modes:

- Can a record be **lost**? (delivered 0 times)
- Can a record be **duplicated**? (delivered  $\geq 2$  times)

Combining the two yields the classical  $2 \times 2$  table:

|                     | No loss             | Possible loss |
|---------------------|---------------------|---------------|
| No duplicates       | <i>Exactly-once</i> | At-most-once  |
| Possible duplicates | At-least-once       | (useless)     |

### 4.2 Mapping to commit strategy

Each box in the table corresponds to a commit strategy:

### 🔑 Consumer Commit Strategy $\Rightarrow$ Delivery Semantics

| Strategy                          | Semantics     |
|-----------------------------------|---------------|
| commit before processing          | at-most-once  |
| commit after processing (default) | at-least-once |
| transactional commit (Kafka + DB) | exactly-once  |

## 4.3 Probability of duplicates under at-least-once

Assume the consumer processes a record and then commits its offset, but crashes between the two operations with probability  $\alpha$  per record. The probability that a record is processed *exactly once* is  $1 - \alpha$ , and the probability it is processed *twice* (re-read after restart) is  $\alpha$ . The expected number of times each record is processed is therefore

$$\mathbb{E}[\text{processings per record}] = 1 \cdot (1 - \alpha) + 2 \cdot \alpha = 1 + \alpha.$$

For  $\alpha = 10^{-3}$  (one crash per thousand records on average), the duplicate rate is 0.1% – negligible for idempotent applications, but catastrophic for, say, a bank transfer.

## 5. Batching: Throughput vs. Latency

### 5.1 Why batching exists

Sending one record per network request is wasteful: each request carries TCP and Kafka protocol overhead. Batching amortises this fixed cost across many records.

Let:

- $\lambda$  = production rate (records / second);
- $b$  = batch size (records);
- $\tau$  = fixed network round-trip time per request (seconds).

### 5.2 Effective throughput

The producer issues  $\lambda/b$  network requests per second. The *network-bound* throughput limit is

$$\lambda_{\max} = \frac{b}{\tau}.$$

Larger batches give higher maximum throughput.

### 🔑 Linger.ms and the Batching Trade-off

The `linger.ms` parameter  $\ell$  caps the time a record waits in the producer's accumulator before being flushed. The expected batch size becomes

$$\mathbb{E}[b] = \min(b_{\max}, \lambda \cdot \ell),$$

where  $b_{\max}$  is set by `batch.size`. Therefore a non-zero `linger.ms` *increases* throughput by allowing fuller batches, at the cost of adding  $\ell$  to the per-record latency.

### 5.3 Numerical illustration

For  $\lambda = 50,000$  rec/s,  $\tau = 5$  ms,  $b_{\max} = 1000$  records:

- `linger.ms` = 0:  $\mathbb{E}[b] \approx 1$ ,  $\lambda_{\max} \approx 200$  rec/s (severely network-bound).
- `linger.ms` = 5:  $\mathbb{E}[b] = \min(1000, 50000 \cdot 0.005) = 250$ ,  $\lambda_{\max} = 50,000$  rec/s.
- `linger.ms` = 20:  $\mathbb{E}[b] = \min(1000, 1000) = 1000$ ,  $\lambda_{\max} = 200,000$  rec/s.

A small `linger.ms` (5–20 ms) typically increases throughput by an order of magnitude.

## 6. Consumer Groups: Partition Assignment Combinatorics

### 6.1 The assignment constraint

In a consumer group, each of the  $P$  partitions is assigned to *exactly one* of the  $C$  consumers. Therefore:

- If  $C \leq P$ : each consumer gets either  $\lfloor P/C \rfloor$  or  $\lceil P/C \rceil$  partitions.
- If  $C > P$ :  $C - P$  consumers stay idle.

#### X<sup>1</sup> Theorem – Effective Parallelism

The number of *active* consumers in a single group is

$$C_{\text{active}} = \min(C, P).$$

### 6.2 Lag dynamics

If the producer rate is  $\lambda$  and each active consumer processes at  $\mu$  records/second, then the aggregate consumer rate is  $C_{\text{active}} \cdot \mu$ . The lag (number of unread records) evolves in time as

$$\frac{dN}{dt} = \lambda - C_{\text{active}} \cdot \mu.$$

- If  $\lambda < C_{\text{active}}\mu$ : the system catches up (lag  $\rightarrow 0$ ).
- If  $\lambda = C_{\text{active}}\mu$ : stable, lag stays constant.
- If  $\lambda > C_{\text{active}}\mu$ : lag grows linearly forever – the system is under-provisioned.

### 6.3 Catch-up time

After a temporary spike that produces backlog  $N_0$ , the time to drain the backlog at steady producer rate  $\lambda$  is

$$T_{\text{catch-up}} = \frac{N_0}{C_{\text{active}}\mu - \lambda}.$$

This formula is the basis of capacity planning for consumer groups.

## 7. Exercises

The two exercises that follow combine the building blocks above. Every numerical value needed is given in the statement.

### Exercise 1 – Producer Reliability and Retries

A team operates a Kafka producer pushing  $\lambda = 100,000$  events per second to a topic with replication factor  $r = 3$ . Each broker has independent failure probability  $p = 0.005$  during any single write window.

#### Part A. Acknowledgements.

- (a) Compute the loss probability for one record under `acks=1`.
- (b) Compute the loss probability under `acks=all` with `min.insync.replicas = 2`.
- (c) Compute the loss probability under `acks=all` with `min.insync.replicas = 3`.
- (d) For each of the three configurations above, compute the *expected number of lost records per second* at  $\lambda = 100,000$  rec/s.

**Part B. Retries.** A transient send error occurs on each individual attempt with probability  $q = 0.04$ . Retries are independent and back-off is  $B = 100$  ms; each attempt itself takes  $L_0 = 10$  ms.

- (e) Compute the expected number of attempts  $\mathbb{E}[T]$  to deliver a record.
- (f) Compute the expected end-to-end latency  $\mathbb{E}[\text{latency}]$ .
- (g) The producer is configured with `retries = R = 5` (so 6 total attempts). Compute the residual loss probability after retries.
- (h) Combine Part A(b) and Part B(c): with `acks=all`, `min.insync.replicas = 2`,  $p = 0.005$ , and the retry policy of (g), what is the overall probability that a record is *ultimately* lost?



 **Exercise 2 – Consumer Groups, Lag, and Delivery Semantics**

An IoT pipeline ingests sensor readings into a Kafka topic with  $P = 12$  partitions. The producer rate is  $\lambda = 60,000$  rec/s. Each consumer in the group can process up to  $\mu = 8,000$  rec/s. The consumer group currently has  $C = 6$  consumers.

**Part A. Capacity and lag.**

- (a) How many partitions does each consumer handle on average?
- (b) Compute the aggregate consumer throughput. Is the system stable?
- (c) A traffic spike doubles the producer rate to  $\lambda' = 120,000$  rec/s for  $T_{\text{spike}} = 30$  seconds. Compute the lag  $N_0$  accumulated by the end of the spike.
- (d) After the spike, the producer rate returns to  $\lambda$ . Compute the time  $T_{\text{catch-up}}$  needed to drain the backlog  $N_0$ .

**Part B. Scaling and rebalancing.** The team considers adding more consumers to handle higher loads.

- (e) What is the maximum useful number of consumers in this group? Why?
- (f) With the maximum useful number of consumers, what is the maximum sustainable producer rate  $\lambda_{\text{max}}$ ?
- (g) The team needs to support  $\lambda^* = 150,000$  rec/s *without* changing  $\mu$ . What is the minimum number of partitions  $P^*$  they should reconfigure the topic with?

**Part C. Delivery semantics.** The application uses at-least-once delivery (commit after processing). The probability that a consumer crashes between processing a record and committing the offset is  $\alpha = 5 \times 10^{-4}$  per record.

- (h) Compute the expected number of times each record is processed.
- (i) At  $\lambda = 60,000$  rec/s, how many duplicate processings occur per hour on average?
- (j) The team is considering switching to exactly-once semantics using transactional commits. Briefly justify in mathematical terms why this would be necessary if the consumer also writes each record to a downstream non-idempotent SQL database.

## 8. Summary

### Key Formulas to Remember

| Concept                          | Formula                                                                                  |
|----------------------------------|------------------------------------------------------------------------------------------|
| Loss prob. <code>acks=1</code>   | $\mathbb{P}(\text{loss}) = p$                                                            |
| Loss prob. <code>acks=all</code> | $\mathbb{P}(\text{loss}) = p^m$ where $m = \text{min.insync.replicas}$                   |
| Expected attempts (geometric)    | $\mathbb{E}[T] = 1/(1 - q)$                                                              |
| Residual loss after $R$ retries  | $q^{R+1}$                                                                                |
| Latency under retries            | $\mathbb{E}[T] \cdot L_0 + (\mathbb{E}[T] - 1) \cdot B$                                  |
| Expected processings             | at-least-once: $1 + \alpha$ ; exactly-once: 1                                            |
| Effective parallelism            | $C_{\text{active}} = \min(C, P)$                                                         |
| Lag dynamics                     | $dN/dt = \lambda - C_{\text{active}}\mu$                                                 |
| Catch-up time                    | $T = N_0/(C_{\text{active}}\mu - \lambda)$                                               |
| Batching: max throughput         | $\lambda_{\text{max}} = b/\tau$ with $\mathbb{E}[b] = \min(b_{\text{max}}, \lambda\ell)$ |

### 8.1 Going further

- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 7–9 (Transactions, Trouble with Distributed Systems, Consistency & Consensus).
- Confluent: “Exactly-Once Semantics in Apache Kafka” (engineering blog, 2017).
- Mitzenmacher & Upfal, *Probability and Computing* (geometric distribution, tail bounds).